# Formal Semantics and Theorem Reference for Mergeable Replicated Data Types

Anonymous Authors

**Abstract**

This document gives a bottom-up reference for the public formal theory of mergeable replicated data types (MRDTs) mechanized in Lean 4. It starts with events and datatype signatures, defines the version-DAG operational semantics, then explains how finite event histories determine stored states, which merge property preserves that explanation, and how operation-generation constraints connect it to an independent client specification. It also covers merge-base construction for version DAGs without a single greatest common ancestor and two independent garbage-collection refinements. Definitions are ordered by semantic dependence rather than Lean file order. Every principal definition and theorem has its exact Lean declaration and a source link. Proof-local lemmas are omitted from the main line; the historical binary development is isolated in an appendix.

## Contents

# 1 Purpose and evidence boundary

This is a reference, not a second narrative paper. It is meant to let a reader reconstruct the formal interface and theorem dependency graph without first learning the layout of the Lean repository. It answers four questions:

1. What are the exact states and transitions of the MRDT machine?
2. What invariant makes every stored version replayable, and what obligation preserves it at merge?
3. How is internal replay connected to an independent client specification?
4. Which history- and state-collection steps refine the same uncollected semantics?

The mathematical displays are transcriptions or notation-level expansions of the cited declarations. The trusted computing base for machine-checked statements is Lean's kernel plus Mathlib. The transcription, notation, prose, and links remain review obligations. No claim is made that a separately implemented runtime is extracted from Lean.

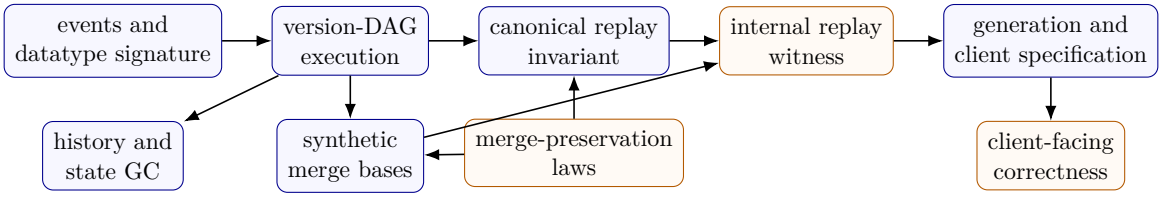


Figure 1: Semantic dependency order. Arrows point from prerequisite to consumer. The two GC layers are optional refinements, not premises of client-facing correctness.

# 2 Primitive types and datatype signatures

## 2.1 Identifiers and events

Replicas, timestamps, and versions are natural numbers:

$$\mathsf{Replica} = \mathbb{N}, \qquad \mathsf{Timestamp} = \mathbb{N}, \qquad \mathsf{Version} = \mathbb{N}.$$

For an application-operation type $O$, an event is

$$\mathsf{Event}(O) = \mathsf{Timestamp} \times \mathsf{Replica} \times O.$$

If $e = (t, r, o)$, write $\mathsf{time}(e) = t$ and $\mathsf{rep}(e) = r$. Timestamps are Lamport timestamps. Global uniqueness and causal monotonicity are properties of valid configurations and the apply transition, not of the bare product type.

*Lean:* `Foundation.Replica, Foundation.Timestamp, Foundation.Op`.

**Definition 2.1** (MRDT signature)**.** An MRDT signature is

$$D = (\Sigma, \sigma_0, O, Q, R, \mathsf{do}, \mathsf{merge}, \mathsf{query})$$

with decidable equality on $\Sigma$ and $O$, and

$$\begin{aligned} \mathsf{do} &: \Sigma \to \mathsf{Event}(O) \to \Sigma, \\ \mathsf{merge} &: \Sigma \to \Sigma \to \Sigma \to \Sigma, \\ \mathsf{query} &: \Sigma \to Q \to R. \end{aligned}$$

The intended reading of $\mathsf{merge}(l, a, b)$ is a three-way merge with ancestor state $l$ and branch states $a, b$. The paper's do is the Lean field `MRDTSig.update`; Lean reserves `do` for its term syntax.

*Lean:* `MRDTSig`.

This signature is the executable datatype interface. Verification attaches certificates to the same $D$ after the replay, merge, generation, and client specifications have been defined. It does not introduce a second datatype signature or a two-way merge.

## 2.2 Finite replay

Finite replay uses only the merge-free projection

$$D_U = (\Sigma, \sigma_0, O, \mathsf{do}).$$

Lean calls this derived structure `UpdateSig`. It is an internal view of $D$, not a second datatype interface.

For $\pi = [e_1, \ldots, e_n]$,

$$\mathsf{applySeq}(D, s, \pi) = \mathsf{do}(\cdots \mathsf{do}(\mathsf{do}(s, e_1), e_2) \cdots, e_n).$$

A list enumerates a set exactly when

$$\mathsf{listPermOf}(\pi, E) \iff \mathsf{Nodup}(\pi) \wedge \forall e.\ (e \in \pi \leftrightarrow e \in E).$$

It respects a relation $R$ when no later element is required to precede an earlier one:

$$\mathsf{respects}(\pi, R) \iff \mathsf{Pairwise}_{\pi}(\lambda a\ b.\ \neg R(b, a)).$$

*Lean:* `Foundation.applySeq, Foundation.listPermOf, Foundation.respects`.

Two events commute when they commute from every state:

$$e_1 \leftrightarrow e_2 \iff \forall s : \Sigma.\ \mathsf{do}(\mathsf{do}(s, e_1), e_2) = \mathsf{do}(\mathsf{do}(s, e_2), e_1).$$

A proof-local replay policy has type

$$\mathsf{order} : \mathsf{Event}(O) \to \mathsf{Event}(O) \to \{\mathsf{FstThenSnd}, \mathsf{SndThenFst}, \mathsf{Either}\}.$$

The default policy returns Either. This policy is used only by the internal replay proof in Section 4. Client-visible interaction ordering is defined independently in Section 7. In Lean, finite replay, commutation, and this proof policy are factored through an internal projection of $D$; the historical reason for that projection is recorded in the final appendix.

*Lean:* `Foundation.UpdateSig.commutes, Foundation.RcRes, Foundation.ReplayPolicy`.

# 3 Version-DAG operational semantics

## 3.1 Graph order

For $P : \mathsf{Version} \to \mathsf{List}(\mathsf{Version})$,

$$\mathsf{Reaches}(P, a, b) \iff (\lambda x\ y.\ x \in P(y))^*(a, b).$$

The ordinary merge rule uses a greatest-common-ancestor predicate:

$$\begin{aligned}\mathsf{IsGCA}(P, v_1, v_2, v_T) \iff\ & v_T\ \mathsf{Reaches}\ v_1 \wedge v_T\ \mathsf{Reaches}\ v_2 \\ & \wedge \forall w.\ w\ \mathsf{Reaches}\ v_1 \to w\ \mathsf{Reaches}\ v_2 \to w\ \mathsf{Reaches}\ v_T.\end{aligned}$$

Every common ancestor reaches a GCA. Ranked reachability is antisymmetric, so a GCA is unique when it exists. This does not say that a DAG always has a unique lowest merge base: a criss-cross DAG may have several incomparable maximal common ancestors and no GCA.

*Lean:* `Reaches, IsGCA`.

*Lean:* `isGCA_unique`.

For a pair of versions $a, b$, define

$$\begin{aligned}\mathsf{CommonAncestors}(P, a, b) &= \{x \mid x\ \mathsf{Reaches}\ a \wedge x\ \mathsf{Reaches}\ b\}, \\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) &\iff m \in \mathsf{CommonAncestors}(P, a, b) \\ &\quad \wedge \forall x \in \mathsf{CommonAncestors}(P, a, b).\ m\ \mathsf{Reaches}\ x \to x = m.\end{aligned}$$

Thus $m$ is maximal when no distinct common ancestor descends from it. The self-case is present because Reaches is reflexive. Several incomparable versions may satisfy this predicate.

*Lean:* `CommonAncestors, IsMaximalCommonAncestor`.

### 3.2 Configurations

For an MRDT $D$, write the version store as a partial map and let $A_C = \mathsf{dom}(S)$ be its allocated-version set. A configuration contains

$$\begin{array}{rcll} S & : & \mathsf{Version} \rightharpoonup (\Sigma \times \mathcal{P}(\mathsf{Event}(O))) & \text{allocated version contents,} \\ H & : & \mathsf{Replica} \rightharpoonup A_C & \text{active replica heads,} \\ P & : & \mathsf{Version} \to \mathsf{List}(\mathsf{Version}) & \text{parent lists,} \\ \mathsf{vis} & : & \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop} & \text{visibility.} \end{array}$$

Thus $C = \langle S, H, P, \mathsf{vis} \rangle$. Write

$$\begin{aligned} \mathsf{state}_C : A_C \to \Sigma, \qquad & \mathsf{state}_C(v) = \mathsf{fst}(S(v)), \\ \mathsf{events}_C : A_C \to \mathcal{P}(\mathsf{Event}(O)), \qquad & \mathsf{events}_C(v) = \mathsf{snd}(S(v)), \\ \mathsf{headState}_C = \mathsf{state}_C \circ H, \qquad & \mathsf{headEvents}_C = \mathsf{events}_C \circ H. \end{aligned}$$

The configuration's active event universe is

$$\begin{aligned} C.\mathsf{events} &\equiv \{e \mid \exists r, E.\ \mathsf{headEvents}_C(r) = E \wedge e \in E\} \\ &= \bigcup_{r \in \mathsf{dom}(H)} \mathsf{headEvents}_C(r). \end{aligned}$$

It contains the events observed at active replica heads, not the union of all historical version event sets. These functions and the active event universe are derived observations, not configuration components.

*Lean:* `Configuration.events`.

For any partial map $f$, write $f(x) = \bot \iff x \notin \mathsf{dom}(f)$.

A well-formed semantic configuration satisfies:

1. Every visibility endpoint occurs in some $\mathsf{headEvents}_C(r)$, and each head event set is closed under visibility.
2. Distinct active events have distinct timestamps; $e_1$ $\mathsf{vis}$ $e_2$ implies $\mathsf{time}(e_1) < \mathsf{time}(e_2)$; and distinct events from one replica are visibility-comparable.
3. Parents have smaller version numbers and $S(0) = (\sigma_0, \varnothing)$.
4. If $S(v_i) = (s_i, E_i)$ for $i \in \{1, 2, T\}$ and $v_T$ is a GCA of $v_1, v_2$, then $E_T = E_1 \cap E_2$.

Thus a configuration is already an operationally well-formed state. Client safety and canonical replay are separate predicates.

*Lean:* `Configuration`.

Lean totalizes the two partial maps: `ver` and `head` return `Option`, while `parents` is the total function $P$. The `head_alloc` field implements the paper type $H : \mathsf{Replica} \rightharpoonup A_C$. The functions `Configuration.headState` and `Configuration.headEvents` implement the two partial compositions above. The store invariant in Section 3.4 connects $P$ to $A_C$: every version named in a parent list must lie in $A_C$. Until that invariant is imposed, keeping $P$ total avoids adding a second allocation domain.

Write $\mathsf{initConfig}(D)$ for the initial configuration. It has replica 0 at version 0, both with state $\sigma_0$ and empty event set. All other replicas and versions are absent, parent lists are empty, and visibility is empty.

*Lean:* `initConfig`.

### 3.3 Labels, freshness, and transition rules

The label constructors have explicit types

$$\begin{aligned} \mathsf{fork} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D), \\ \mathsf{apply} &: \mathsf{Timestamp} \to \mathsf{Replica} \to O \to \mathsf{Label}(D), \\ \mathsf{merge} &: \mathsf{Replica} \to \mathsf{Replica} \to \mathsf{Label}(D), \\ \mathsf{query} &: \mathsf{Replica} \to Q \to R \to \mathsf{Label}(D). \end{aligned}$$

Fork is destination-first: $\mathsf{fork}(r_d, r_s)$.

*Lean:* `Label`.

Write $\mathsf{freshTime}(C, t)$ when no event at an allocated version has timestamp $t$, and $\mathsf{freshVer}(C, v')$ when $v' \notin A_C$. The condition $v < v'$ is a rank condition for the version DAG, not a timestamp comparison. It makes the parent relation well founded. The following rules display all changed semantic fields; other entries remain unchanged, and the resulting configuration satisfies the configuration conditions above.

Fork

$$\frac{\begin{array}{c} H(r_s) = v \quad \mathsf{headState}_C(r_s) = s \quad \mathsf{headEvents}_C(r_s) = E \\ H(r_d) = \bot \quad \mathsf{freshVer}(C, v') \quad v < v' \end{array}}{C \xrightarrow{\mathsf{fork}(r_d, r_s)} C\big[S(v') \mapsto (s, E), \quad H(r_d) \mapsto v', \quad P(v') \mapsto [v] \quad \big]}$$

Fork creates a fresh child of an existing source head. It neither resets to the root nor creates a disconnected history.

*Lean:* `Step.fork`.

*Lean:* `Step.fork_copies_source, Step.fork_head_ne_root`.

Apply

$$\frac{\begin{array}{c} H(r) = v \quad \mathsf{headState}_C(r) = s \quad \mathsf{headEvents}_C(r) = E \quad e = (t, r, o) \\ \mathsf{freshTime}(C, t) \quad \mathsf{freshVer}(C, v') \quad v < v' \end{array}}{C \xrightarrow{\mathsf{apply}(t,r,o)} C \begin{bmatrix} S(v') \mapsto (\mathsf{do}(s, e), E \cup \{e\}), & H(r) \mapsto v', \\ P(v') \mapsto [v], & \mathsf{vis} \mapsto \mathsf{vis} \cup (E \times \{e\}) \end{bmatrix}}$$

Freshness gives globally unique timestamps. Because the resulting configuration also satisfies causal timestamp monotonicity, all events visible to $e$ have smaller timestamps. Together these are the Lamport-clock conditions. The raw rule has no issuance premise.

*Lean:* `Step.apply`.

Query

$$\frac{\mathsf{headState}_C(r) = s \qquad z = \mathsf{query}(s, q)}{C \xrightarrow{\mathsf{query}(r,q,z)} C}$$

Query is observational.

*Lean:* `Step.query`.

MERGE

$$\frac{\begin{array}{c} H(r_i) = v_i \quad \mathsf{headState}_C(r_i) = s_i \quad \mathsf{headEvents}_C(r_i) = E_i \qquad (i \in \{1,2\}) \\ \mathsf{IsGCA}(P, v_1, v_2, v_T) \quad \mathsf{state}_C(v_T) = s_T \quad \mathsf{events}_C(v_T) = E_T \\ \mathsf{freshVer}(C, v_m) \quad v_1 < v_m \quad v_2 < v_m \end{array}}{C \xrightarrow{\mathsf{merge}(r_1,r_2)} C \begin{bmatrix} S(v_m) \mapsto (\mathsf{merge}(s_T, s_1, s_2), E_1 \cup E_2), & \\ H(r_1) \mapsto v_m, & P(v_m) \mapsto [v_1, v_2] \end{bmatrix}}$$

The first replica receives the merged head. Merge creates no event.

*Lean:* `Step.merge`.

### 3.4 Store invariant and GCA events

The store invariant has three components. First, define

$$\mathsf{ParentsAllocated}(S, P) \iff \forall v, p.\ p \in P(v) \Rightarrow p \in \mathsf{dom}(S).$$

Every parent named by an edge is present in the version store. Under this condition, $P$ may be viewed as $P : \mathsf{Version} \to \mathsf{List}(A_C)$, and its restriction to allocated children has type $P|_{A_C} : A_C \to \mathsf{List}(A_C)$. Define event monotonicity along a parent edge by

$$\begin{aligned} \mathsf{EventsMonotone}(S, P) \iff \forall v, p, s_p, E_p, s_v, E_v.\ p \in P(v) \wedge S(p) = (s_p, E_p) \wedge S(v) = (s_v, E_v) \\ \Rightarrow E_p \subseteq E_v. \end{aligned}$$

Thus a child contains every event stored at its parent. Finally, define

$$\begin{aligned} \mathsf{EventOrigin}(S, P) \iff \forall v, s, E, e.\ S(v) = (s, E) \wedge e \in E \Rightarrow \\ \exists v_0, s_0, E_0.\ S(v_0) = (s_0, E_0) \wedge e \in E_0 \\ \wedge \forall w, s_w, E_w.\ S(w) = (s_w, E_w) \wedge e \in E_w \\ \Rightarrow \mathsf{Reaches}(P, v_0, w). \end{aligned}$$

Every stored event therefore has an allocated origin version containing it; that version is an ancestor of every allocated version containing the event. No uniqueness of the origin is asserted. The store invariant is the conjunction

$$\begin{aligned} \mathsf{StoreInv}(S, P) \iff \mathsf{ParentsAllocated}(S, P) \\ \wedge\ \mathsf{EventsMonotone}(S, P) \\ \wedge\ \mathsf{EventOrigin}(S, P). \end{aligned}$$

These three predicates correspond respectively to Lean's `parents_alloc`, `events_mono`, and `origin` fields. The proof of the GCA event equation factors through their conjunction. Its principal consequence is

$$\begin{aligned} \mathsf{StoreInv}(S, P) \wedge \mathsf{IsGCA}(P, v_1, v_2, v_T) \\ \wedge S(v_i) = (s_i, E_i)\ (i \in \{1, 2, T\}) \Longrightarrow E_T = E_1 \cap E_2. \end{aligned}$$

*Lean:* `StoreInv, gca_events_of_storeInv`.

# 4 Replay contexts and set-relative order

## 4.1 Replay context

For a stored event set $E$, the replay problem is to enumerate exactly the events in $E$ in an admissible order and fold their updates from $\sigma_0$. Admissibility depends on which events have been observed and on their visibility relation. It does not depend on version identifiers, parent edges, materialized states, or merge bases. The replay context isolates this smaller input to the ordering proofs.

Let $L : \mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O))$ map each active replica to the event set at its current head. Here $a$ $\mathsf{vis}$ $b$ means that $a$ is visible to $b$, so $a$ causally precedes $b$. Define the observed event universe by

$$\mathsf{Observed}_L(e) \iff \exists r, E.\ L(r) = E \wedge e \in E.$$

The two required coherence conditions are

$$\begin{aligned}\mathsf{TimestampUnique}(L) &\iff \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b \Rightarrow \mathsf{time}(a) \neq \mathsf{time}(b),\\ \mathsf{SameReplicaTotal}(L, \mathsf{vis}) &\iff \forall a, b.\ \mathsf{Observed}_L(a) \wedge \mathsf{Observed}_L(b) \wedge a \neq b\\ &\qquad \wedge \mathsf{rep}(a) = \mathsf{rep}(b) \Rightarrow a\ \mathsf{vis}\ b \vee b\ \mathsf{vis}\ a.\end{aligned}$$

Thus timestamps identify observed events globally, while visibility compares every pair of distinct events issued by one replica. A replay context is formally

$$\mathsf{ReplayContext}(D) = \left\langle \begin{array}{l} L : \mathsf{Replica} \rightharpoonup \mathcal{P}(\mathsf{Event}(O)),\\ \mathsf{vis}\colon \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Prop},\\ u_{\mathrm{ts}} : \mathsf{TimestampUnique}(L),\\ u_{\mathrm{rep}} : \mathsf{SameReplicaTotal}(L, \mathsf{vis}) \end{array} \right\rangle.$$

For a replay context $R = \langle L, \mathsf{vis}, u_{\mathrm{ts}}, u_{\mathrm{rep}} \rangle$, the event universe is the union of the active replicas' head-event sets:

$$R.\mathsf{events} = \{e \mid \mathsf{Observed}_L(e)\} = \bigcup_{r \in \mathsf{dom}(L)} L(r).$$

Every MRDT configuration $C$ projects to a replay context by taking $L = \mathsf{headEvents}_C$ and retaining $C.\,\mathsf{vis}$. The configuration fields `timestamps_distinct` and `vis_total_same_replica` supply the two proofs. Lean totalizes $L$ as an Option-valued function; `ReplayContext.L` receives the derived head-event map, and `Configuration.replayContext` constructs the projection. Write $R_C = \mathsf{replayContext}(C)$ for this projection whenever $C$ is an MRDT configuration. The projection preserves the active event universe:

$$R_C.\mathsf{events} = C.\mathsf{events}.$$

*Lean:* `Foundation.ReplayContext`.

*Lean:* `Configuration.replayContext`.

## 4.2 Set-relative replay order

For replay context $R$, version event set $E$, and events $e_1, e_2$, define

$$\begin{aligned}e_1\ \mathsf{loOn}_{R,E}\ e_2 \iff &(e_1\ \mathsf{vis}\ e_2 \wedge \neg(e_1 \leftrightarrow e_2))\\ &\vee (\neg(e_1\ \mathsf{vis}\ e_2) \wedge \neg(e_2\ \mathsf{vis}\ e_1) \wedge \mathsf{order}(e_1, e_2) = \mathsf{FstThenSnd}\\ &\quad \wedge \neg\exists e_3 \in E.\ e_2\ \mathsf{vis}\ e_3 \wedge \neg(e_2 \leftrightarrow e_3)).\end{aligned}$$

An event $e_3 \in E$ satisfying the final existential condition is an *absorber* for $e_2$. Only an absorber inside $E$ can remove an arbitration edge needed to replay $E$. Growing the set may add absorbers and therefore remove such edges:

$$E \subseteq E' \wedge \mathsf{loOn}_{R,E'}(a,b) \Longrightarrow \mathsf{loOn}_{R,E}(a,b).$$

The historical global order is $\mathsf{lo}_R = \mathsf{loOn}_{R,R.\mathsf{events}}$; every $\mathsf{lo}_R$ edge is an edge of $\mathsf{loOn}_{R,E}$.

*Lean:* `Foundation.loOn, Foundation.loOn_mono, Foundation.loOn_of_lo`.

### 4.3 Replay laws and convergence

Write

$$\mathsf{distinct}(a,b) \Longleftrightarrow \mathsf{time}(a) \neq \mathsf{time}(b),$$
$$\mathsf{differentReplicas}(a,b) \Longleftrightarrow \mathsf{rep}(a) \neq \mathsf{rep}(b).$$

Thus distinct is timestamp inequality, which implies event inequality for observed events because replay contexts have unique timestamps. The update-layer bundle has three conjuncts. Directional noncommutation is

$$\mathsf{RcNonCommDirectional}(D) \Longleftrightarrow \forall a, b.\ \mathsf{distinct}(a,b) \wedge \mathsf{differentReplicas}(a,b) \Longrightarrow \left( \begin{array}{rl} \neg(a \leftrightarrow b) \Longleftrightarrow & (\mathsf{order}(a,b) = \mathsf{FstThenSnd}) \\ & \vee\ (\mathsf{order}(b,a) = \mathsf{FstThenSnd}) \end{array} \right).$$

Thus every distinct cross-replica conflict is oriented in at least one direction, and only conflicts receive such an orientation. The chain restriction is

$$\mathsf{NoRcChain}(D) \Longleftrightarrow \forall a, b, c.\ \mathsf{distinct}(a,b) \wedge \mathsf{distinct}(b,c) \Longrightarrow \neg\big(\mathsf{order}(a,b) = \mathsf{FstThenSnd} \wedge \mathsf{order}(b,c) = \mathsf{FstThenSnd}\big).$$

It rules out two consecutive first-then-second policy edges. Finally, conditional commutation lift is

$$\begin{aligned} \mathsf{CondCommLift}(D) \Longleftrightarrow \forall s, e, e', e'', \pi.\ & \mathsf{distinct}(e,e') \wedge \mathsf{distinct}(e,e'') \wedge \mathsf{distinct}(e',e'') \\ & \wedge \mathsf{order}(e,e') = \mathsf{FstThenSnd} \wedge \neg(e' \leftrightarrow e'') \\ & \Longrightarrow \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s,e'),e),\pi),e'') \\ & \quad = \mathsf{do}(\mathsf{applySeq}(D, \mathsf{do}(\mathsf{do}(s,e),e'),\pi),e''). \end{aligned}$$

Therefore

$$\mathsf{ReplayLaws}(D) \Longleftrightarrow \mathsf{RcNonCommDirectional}(D) \wedge \mathsf{NoRcChain}(D) \wedge \mathsf{CondCommLift}(D).$$

These conjuncts correspond respectively to Lean's `rc_non_comm_directional`, `no_rc_chain`, and `cond_comm_lift` fields.

*Lean:* `ReplayLaws`.

With these laws, transitive and irreflexive visibility, and finite support, $\mathsf{loOn}_{R,E}$ is acyclic on $E$, admits a respecting enumeration, and all respecting enumerations fold to the same state. Intuitively, acyclicity says that the replay constraints are jointly schedulable; existence produces at least one such schedule; and convergence says that choosing a different valid schedule cannot change the result. Thus a finite event set $E$ determines one replayed state even when it admits several valid orders.

For events in $E$, write

$$a \ \mathsf{loOn}^{\neq}_{R,E} \ b \iff a \neq b \wedge a \in E \wedge b \in E \wedge a \ \mathsf{loOn}_{R,E} \ b.$$

Write $\mathsf{TransGen}(Q)$ for the nonempty transitive closure of a relation $Q$.

**Lemma 4.1** (Existence and uniqueness of set-relative replay)**.** *Suppose*

$$\mathsf{ReplayLaws}(D), \qquad \mathsf{Transitive}(R.\mathsf{vis}), \qquad \mathsf{Irreflexive}(R.\mathsf{vis}),$$
$$E \subseteq R.\mathsf{events}, \qquad \mathsf{listPermOf}(\ell, E).$$

*Then:*
*1.* $\mathsf{loOn}^{\neq}_{R,E}$ *is acyclic:*

$$\forall a.\ \neg \mathsf{TransGen}(\mathsf{loOn}^{\neq}_{R,E})(a, a).$$

*2. E has a respecting enumeration:*

$$\exists \rho.\ \mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E}).$$

*3. all respecting enumerations of E have the same fold, from every starting state:*

$$\begin{aligned} \forall s, \rho_1, \rho_2. \quad & \mathsf{listPermOf}(\rho_1, E) \wedge \mathsf{listPermOf}(\rho_2, E) \\ & \wedge \mathsf{respects}(\rho_1, \mathsf{loOn}_{R,E}) \wedge \mathsf{respects}(\rho_2, \mathsf{loOn}_{R,E}) \\ & \Longrightarrow \mathsf{applySeq}(D, s, \rho_1) = \mathsf{applySeq}(D, s, \rho_2). \end{aligned}$$

*Proof sketch.* For acyclicity, first observe that a policy edge cannot have a successor in $\mathsf{loOn}_{R,E}$. If the successor is a visibility edge, its endpoint is an absorber forbidden by the policy edge; if it is another policy edge, NoRcChain gives a contradiction. It follows inductively that every nonempty transitive $\mathsf{loOn}^{\neq}_{R,E}$ path is either a visibility path or ends with a policy edge that cannot be extended. A cycle of the first kind would give $a$ vis $a$ by transitivity, contradicting irreflexivity. A cycle of the second kind would extend its terminal policy edge by the first edge of the cycle, also a contradiction.

For existence, use $\ell$ as a finite enumeration of $E$. A walk through this list must find a $\mathsf{loOn}_{R,E}$-maximal event; otherwise following outgoing edges would revisit an event and create a cycle. Remove that event, recursively enumerate the remaining set, and append the maximal event. Repeating this construction produces a permutation that respects $\mathsf{loOn}_{R,E}$.

For uniqueness, induct on the length of one respecting enumeration. Let $e$ be its head and locate $e$ in the other enumeration, say $\rho_2 = \sigma +\!\!+ [e] +\!\!+ \tau$. Minimality of $e$ in the first enumeration and respectfulness of the second make $e$ incomparable with every event crossed while moving it left through $\sigma$. A commuting event swaps with $e$ directly. For a noncommuting event $y$, same-replica visibility totality would make $y$ and $e$ comparable, so they must come from different replicas. RcNonCommDirectional then orients their conflict. Since that orientation nevertheless creates no $\mathsf{loOn}_{R,E}$ edge, the definition of loOn supplies a later absorber $e_3 \in E$. Respectfulness places the absorber after the crossed pair, and CondCommLift shows that swapping $y$ and $e$ does not change the state after $e_3$ is applied. Bubble $e$ to the front, remove it from both enumerations, and apply the induction hypothesis. The strengthened induction retains absorbers of surviving events, which is why no backward-closure premise on $E$ is required. □

*Lean:* `loOnNe_acyclic_of_replayLaws`, `exists_loOn_respecting_perm_of_replayLaws`, `convergence_on_of_replayLaws`.

# 5 Canonical states and canonical configurations

**Definition 5.1** (Canonical state)**.**

$$\mathsf{Canonical}(R, E, s) \iff \exists \rho.\ \mathsf{listPermOf}(\rho, E) \wedge \mathsf{respects}(\rho, \mathsf{loOn}_{R,E}) \\ \wedge\ \mathsf{applySeq}(D, \sigma_0, \rho) = s.$$

By set-relative convergence, the witness denotes the unique fold of every respecting enumeration, under the replay laws.

*Lean:* `Foundation.IsCanonicalState`.

**Definition 5.2** (Canonical configuration)**.** Let $R_C = \mathsf{replayContext}(C)$. Then $\mathsf{CanonicalConfig}(C)$ consists of:

| | |
|---|---|
| canonical : | $S(v) = (s, E) \Rightarrow \mathsf{Canonical}(R_C, E, s)$, |
| visTrans : | $a\ \mathsf{vis}\ b \wedge b\ \mathsf{vis}\ c \Rightarrow a\ \mathsf{vis}\ c$, |
| visIrrefl : | $\neg(a\ \mathsf{vis}\ a)$, |
| supported : | $S(v) = (s, E) \wedge a \in E \Rightarrow a \in C.\mathsf{events}$, |
| causal : | $S(v) = (s, E) \wedge a\ \mathsf{vis}\ b \wedge b \in E \Rightarrow a \in E$, |

*Lean:* `CanonicalConfig`.

The invariant ranges over every allocated version, including historical merge bases. Its internal replay projection is

$$\mathsf{HasReplayWitness}(C) \iff \forall v, s, E.\ S(v) = (s, E) \to \exists \pi.\ \mathsf{listPermOf}(\pi, E) \\ \wedge\ \mathsf{respects}(\pi, \mathsf{lo}_C) \\ \wedge\ \mathsf{applySeq}(D, \sigma_0, \pi) = s.$$

Canonicality implies this property because respecting the stronger set-relative order implies respecting the global order:

$$\mathsf{CanonicalConfig}(C) \Longrightarrow \mathsf{HasReplayWitness}(C).$$

This is internal replay adequacy, not client-facing correctness.

*Lean:* `HasReplayWitness, hasReplayWitness_of_canonical`.

# 6 Merge preservation

## 6.1 The semantic contract

For replay context $R$, abbreviate

$$\mathsf{Context}(R) \equiv \mathsf{Transitive}(\mathsf{vis}) \wedge \mathsf{Irreflexive}(\mathsf{vis}), \\ \mathsf{Supported}_R(E) \equiv E \subseteq R.\mathsf{events}, \\ \mathsf{WeakClosed}_R(E) \equiv \forall a, b.\ a\ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b) \wedge b \in E \to a \in E.$$

**Definition 6.1** (Join at a context)**.** `JoinAt D R` is

$$\forall E_1, E_2, s_0, s_1, s_2.\ \mathsf{Context}(R) \wedge \bigwedge_{i=1,2} (\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i)) \\ \wedge\ \mathsf{Canonical}(R, E_1 \cap E_2, s_0) \wedge \mathsf{Canonical}(R, E_1, s_1) \wedge \mathsf{Canonical}(R, E_2, s_2) \\ \Longrightarrow \mathsf{Canonical}(R, E_1 \cup E_2, \mathsf{merge}(s_0, s_1, s_2)).$$

`Join D` is $\forall R.$ `JoinAt D R`.

*Lean:* `JoinAt, Join`.

Join is the load-bearing merge theorem: canonical intersection and branch states map to the canonical union state. It mentions neither reachability nor a particular version graph.

**Definition 6.2** (Restricted merge scope)**.** For a predicate $G : \mathsf{ReplayContext}(D) \to \mathsf{Prop}$,

$$\mathsf{JoinOn}(D, G) \iff \forall R.\ G(R) \to \mathsf{JoinAt}(D, R).$$

Thus all-context `Join` has no configuration precondition, whereas `JoinOn` records one explicitly without saying how executions establish it. All-context Join supplies Join under every predicate, and $\mathsf{JoinOn}(D, \lambda\_.\mathsf{True})$ is equivalent to $\mathsf{Join}(D)$.

*Lean:* `JoinOn, Join.toJoinOn, joinOn_true_iff`.

These are the complete semantic interface for merge preservation. The remaining definitions in this section are reusable ways to prove all-context `Join`; they do not introduce further notions of merge correctness.

## 6.2 Proof routes for merge preservation

The reusable proof has one primary canonical-state contract. A datatype may prove that contract directly or obtain it from stronger equations over arbitrary states. It may also bypass the reusable induction and prove `Join` or `JoinOn` directly.

### 6.2.1 Canonical-state Join laws

For any nonempty finite family of event sets, write

$$\mathsf{Admissible}_R(E_1, \ldots, E_n) \equiv \mathsf{Context}(R) \wedge \bigwedge_{i=1}^{n} (\mathsf{Supported}_R(E_i) \wedge \mathsf{WeakClosed}_R(E_i)),$$
$$\mathsf{Maximal}_{R,U}(e) \equiv e \in U \wedge \forall x \in U.\ x \neq e \Rightarrow \neg \mathsf{IoOn}_{R,U}(e, x).$$

Define the one-step noncommuting-visibility relation by $\mathsf{visNC}_R(a, b) \equiv a \ \mathsf{vis}\ b \wedge \neg(a \leftrightarrow b)$, and write $\mathsf{visNC}_R^+$ for its positive transitive closure. The principal downset of $e$ is

$$\downarrow_R e \equiv \{x \mid x = e \vee \mathsf{visNC}_R^+(x, e)\}.$$

We omit the subscript $R$ below when the replay context is fixed. In particular, if $e \in E$ and $\mathsf{WeakClosed}_R(E)$, then $\downarrow_R e \subseteq E$.

The common core is

$$\mathsf{JoinCoreLaws}(D) = \langle \mathsf{replay} : \mathsf{ReplayLaws}(D),\ \mathsf{mergeComm} : \forall l, a, b.\ \mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a) \rangle.$$

Thus `JoinCoreLaws` contains replay and branch commutativity.

*Lean:* `JoinCoreLaws`.

The canonical delta component, `FeasibleDeltaLaws`, contains a unit law and the two delta equations restricted to the exact canonical tuples consumed by the Join induction. Below, $R$

ranges over replay contexts, $E, E_1, E_2, U$ over event sets, $e$ over events, and $s, l, m, x, y, x_0, x_1, x_2$ over states.

$$\textsf{init}: \quad \forall R, E, s. \quad \textsf{Supported}_R(E) \wedge \textsf{Canonical}(R, E, s) \implies \textsf{merge}(\sigma_0, \sigma_0, s) = s.$$

Thus the unit equation is required only for a state produced by a canonical replay of a supported event set.

For an event present only on the first branch,

$$\begin{aligned}
&\textsf{localRedistribute}: \\
&\forall R, E_1, E_2, l, m, x, y, e. \\
&\textsf{Admissible}_R(E_1, E_2) \wedge e \in E_1 \setminus E_2 \wedge \textsf{Maximal}_{R, E_1 \cup E_2}(e) \\
&\wedge \textsf{Canonical}(R, E_1 \cap E_2, l) \wedge \textsf{Canonical}(R, \downarrow e \setminus \{e\}, m) \\
&\wedge \textsf{Canonical}(R, E_1 \setminus \{e\}, x) \wedge \textsf{Canonical}(R, E_2, y) \\
&\implies \textsf{merge}(l, \textsf{merge}(m, x, \textsf{do}(m, e)), y) \\
&\qquad = \textsf{merge}(m, \textsf{merge}(l, x, y), \textsf{do}(m, e)).
\end{aligned}$$

Intuitively, the equation extracts the maximal local event through the outer merge. The states $l$, $m$, $x$, and $y$ represent respectively the branch intersection, the noncommuting dependency past of $e$, the first branch without $e$, and the second branch.

Figure 2 presents the two sides of `localRedistribute` as alternative factorizations of the same union history.

For an event shared by both branches,

$$\begin{aligned}
&\textsf{redistribute}: \\
&\forall R, E_1, E_2, m, x_0, x_1, x_2, e. \\
&\textsf{Admissible}_R(E_1, E_2) \wedge e \in E_1 \cap E_2 \wedge \textsf{Maximal}_{R, E_1 \cup E_2}(e) \\
&\wedge \textsf{Canonical}(R, (E_1 \cap E_2) \setminus \{e\}, x_0) \wedge \textsf{Canonical}(R, \downarrow e \setminus \{e\}, m) \\
&\wedge \textsf{Canonical}(R, E_1 \setminus \{e\}, x_1) \wedge \textsf{Canonical}(R, E_2 \setminus \{e\}, x_2) \\
&\implies \textsf{merge}(\textsf{merge}(m, x_0, \textsf{do}(m, e)), \textsf{merge}(m, x_1, \textsf{do}(m, e)), \textsf{merge}(m, x_2, \textsf{do}(m, e))) \\
&\qquad = \textsf{merge}(m, \textsf{merge}(x_0, x_1, x_2), \textsf{do}(m, e)).
\end{aligned}$$

This equation extracts one shared maximal event from all three merge components. The feasibility restrictions exclude representation states that no event history constructs.

*Lean:* `FeasibleDeltaLaws`.

Figure 3 presents the two sides of `redistribute` as alternative factorizations of the same union history.

The last component peels the selected maximal event from the target union.

$$\begin{aligned}
\textsf{CausalDeltaLaw}(D) \iff \forall R, U, A, B, e. \quad & \textsf{Admissible}_R(U) \wedge \textsf{Maximal}_{R,U}(e) \\
& \wedge \textsf{Canonical}(R, U \setminus \{e\}, A) \wedge \textsf{Canonical}(R, \downarrow e \setminus \{e\}, B) \\
& \implies \textsf{merge}(B, A, \textsf{do}(B, e)) = \textsf{do}(A, e).
\end{aligned}$$

Here $A$ represents the remaining history and $B$ represents the event's noncommuting dependency past.

*Lean:* `CausalDeltaLaw`.

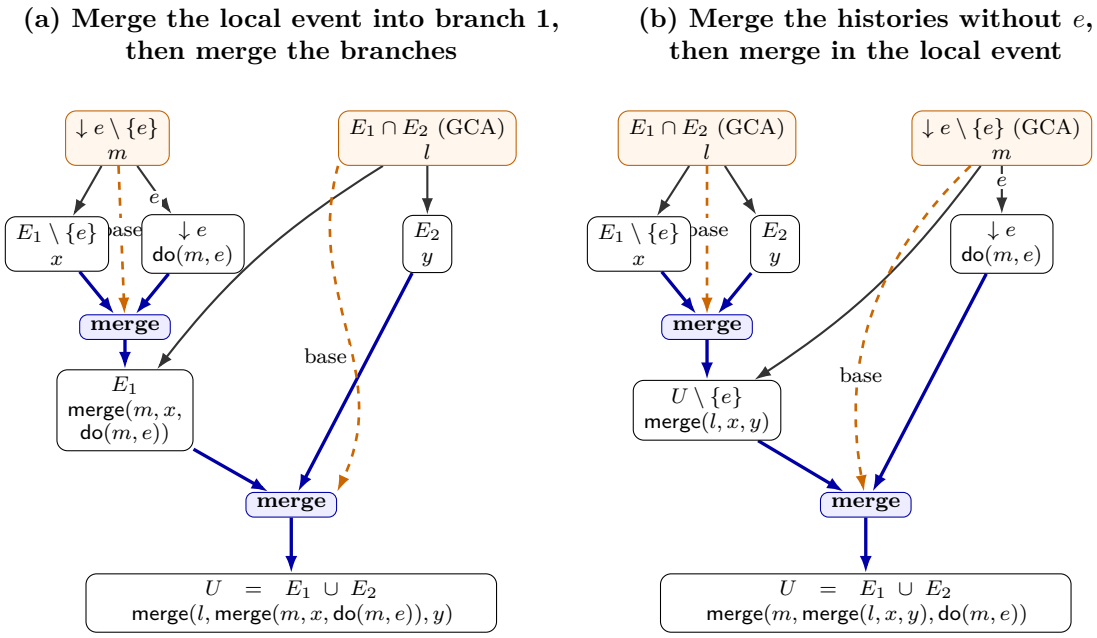


Figure 2: The local redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state to a gate, and orange boxes identify those GCA states. All boxes are proof-level canonical states; the law does not require all of them to be allocated versions in one execution. Panel (a) constructs the $E_1$ state before the outer merge. Panel (b) first constructs the $U \setminus \{e\}$ state and then merges in $e$. Both panels flow from top to bottom. `localRedistribute` identifies the two terminal states.

The single instance-facing bundle is

$$\begin{aligned}\mathsf{CanonicalJoinLaws}(D) = \langle &\mathsf{core} : \mathsf{JoinCoreLaws}(D),\\ &\mathsf{delta} : \mathsf{FeasibleDeltaLaws}(D),\\ &\mathsf{causalDelta} : \mathsf{CausalDeltaLaw}(D)\rangle.\end{aligned}$$

Its primary theorem is

$$\mathsf{CanonicalJoinLaws}(D) \Longrightarrow \mathsf{Join}(D).$$

The proof inducts on the union history. The empty case uses the canonical unit law. A maximal event local to one branch uses `localRedistribute`; a maximal event shared by both branches uses `redistribute`. Both cases use `CausalDeltaLaw` to peel the selected event.

*Lean:* `CanonicalJoinLaws`.

*Lean:* `CanonicalJoinLaws.join, JoinProof.ofFeasibleStateLaws`.

### 6.2.2 Universal equations as a shortcut

Many datatypes satisfy stronger equations for arbitrary representation states. The universal merge bundle `MergeLaws` contains only the fields used by the universal-to-canonical bridge:

$$\begin{aligned}\mathsf{MergeLaws}(D) = \langle &\mathsf{replay} : \mathsf{ReplayLaws}(D),\\ &\mathsf{mergeComm} : \forall l, a, b.\ \mathsf{merge}(l, a, b) = \mathsf{merge}(l, b, a),\\ &\mathsf{mergeInit} : \forall s.\ \mathsf{merge}(\sigma_0, \sigma_0, s) = s\rangle.\end{aligned}$$

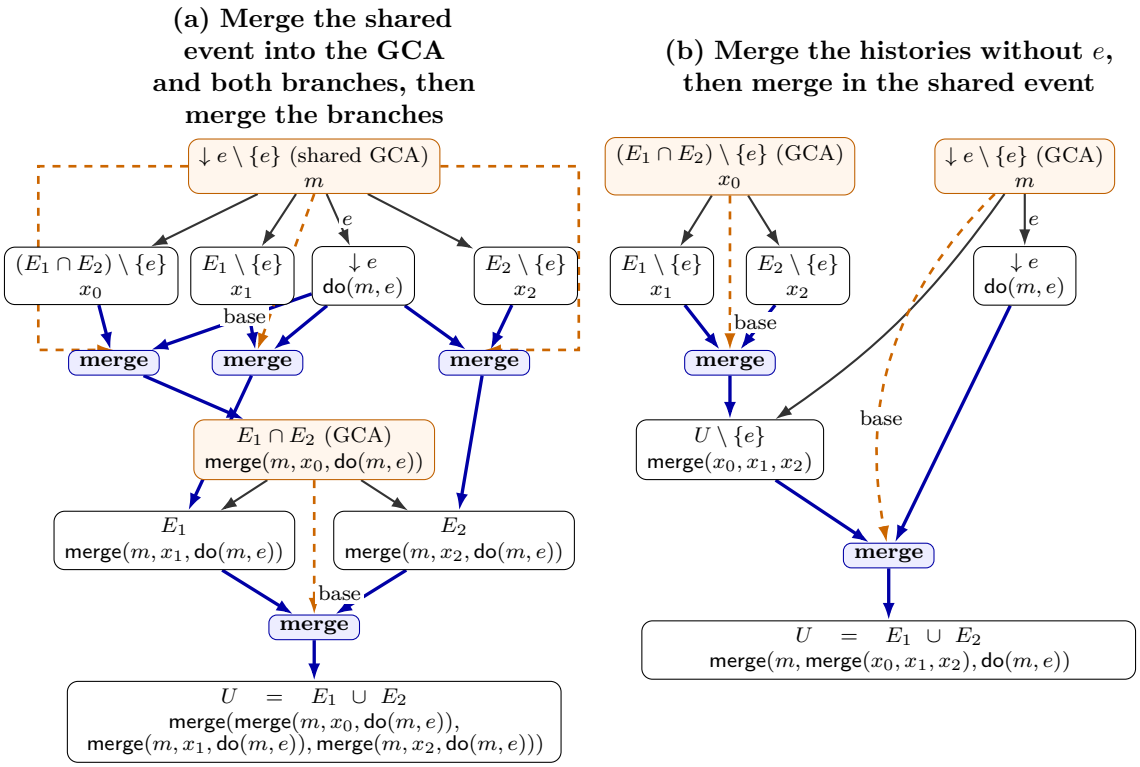


Figure 3: The shared redistribution law. Each box pairs an abstract event set (top) with its canonical state (bottom). Black arrows show event-history extension. Thick blue arrows enter and leave explicit merge gates. Orange dashed arrows supply the GCA state, and orange boxes identify GCA states. Panel (a) uses the same state $m$ as the base of three inner merges, producing the branch intersection and the two branch states before the outer merge. Panel (b) first constructs $U \setminus \{e\}$ at base $x_0$, then merges it with $\downarrow e$ at base $m$. The fan-out edges in panel (a) reuse the same $m$ and $\mathsf{do}(m, e)$ states. Both panels flow from top to bottom, and `redistribute` identifies their terminal states.

*Lean:* `MergeLaws`.

The bundle `DeltaLaws` contains the two universal delta equations. Here $c$ is an arbitrary state; the canonical adapter substitutes $\mathsf{do}(m, e)$ for it.

$$\mathsf{merge}(\mathsf{merge}(m, x_0, c), \mathsf{merge}(m, x_1, c), \mathsf{merge}(m, x_2, c)) = \mathsf{merge}(m, \mathsf{merge}(x_0, x_1, x_2), c),$$
$$\mathsf{merge}(l, \mathsf{merge}(m, x, c), y) = \mathsf{merge}(m, \mathsf{merge}(l, x, y), c).$$

*Lean:* `DeltaLaws`.

The auxiliary `CommutingPeelLaw` is not part of the canonical Join contract. Together with `MergeLaws` and universal update commutativity, it supplies `CausalDeltaLaw` for the all-commuting instance class.

$$\mathsf{MergeLaws}(D) + \mathsf{CommutingPeelLaw}(D) + (\forall e, e'.\ e \leftrightarrow e') \Longrightarrow \mathsf{CausalDeltaLaw}(D).$$

*Lean:* `CommutingPeelLaw`.

*Lean:* `causalDeltaLaw_of_all_comm`.

The universal equations construct the canonical contract:

$$\begin{aligned}\mathsf{MergeLaws}(D) &\Longrightarrow \mathsf{JoinCoreLaws}(D),\\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) &\Longrightarrow \mathsf{FeasibleDeltaLaws}(D),\\ \mathsf{MergeLaws}(D) \wedge \mathsf{DeltaLaws}(D) \wedge \mathsf{CausalDeltaLaw}(D) &\Longrightarrow \mathsf{CanonicalJoinLaws}(D).\end{aligned}$$

*Lean:* `MergeLaws.toJoinCore`.

*Lean:* `feasibleDeltaLaws_of_delta, CanonicalJoinLaws.ofArbitrary`.

The bridge uses `MergeLaws` for the common core and canonical unit law, substitutes $\mathsf{do}(m, e)$ for the arbitrary delta-state argument in each `DeltaLaws` equation, and discards the canonical-tuple premises.

Consequently, `JoinProof.ofArbitraryStateLaws` is only the composition of `CanonicalJoinLaws.ofArbitrary` and `CanonicalJoinLaws.join`. It does not define a second Join property or a second foundational induction.

*Lean:* `JoinProof.ofArbitraryStateLaws`.

### 6.2.3 Instance use

The semantic audit lists each RDT family first and keeps merge scope separate from operation issuance.

| RDT | Merge theorem | Issuance policy | Role of issuance |
|---|---|---|---|
| Grow-only stores and flat counters | Join for all contexts | `True` | No issuance restriction is needed. |
| OR-Set | Join for all contexts | `canIssue` | Establishes refinement from tagged storage to the ordinary add-wins set, not Join. |
| Bounded counter | Join for all contexts | `bcApplicable` | Establishes rights-respecting sequential legality and safety, not Join. |
| TreeMove | Join for all contexts | `applicable` | Restricts issuer inputs to visible nodes and reserved-node rules; the total chronological sequential proof does not require it. |
| AegisSheet | Join for all contexts | `applicable` | Establishes causal-origin legality and safety, not Join. |
| RGA | Join for all contexts | `applicable` | Establishes ordinary-list legality, not Join. |
| Multi-value register | Join for all contexts | `mvrApplicable` | Establishes the raw-fold sequential premise, not Join. |
| Queue | Join under `QHonestCore` | `qApplicable` | Establishes the Join condition and sequential legality. |
| EmbedRGA | Join under `EHonestCore` | `eApplicable` | Establishes the Join condition and ordinary-list legality. |
| SidedEmbedRGA | Join under `SHonestCore` | `sApplicable` | Establishes the Join condition and ordinary-list legality. |

| RDT | Merge theorem | Issuance policy | Role of issuance |
|---|---|---|---|
| FugueMax proof signature | Join under `FMHonestCore` | `fApplicable` | Establishes the Join condition; this is an internal proof signature. |
| SidedPeritext | Join under projected `SHonestCore` | `coreGuard` | Establishes the sequence Join condition and rich-editor legality. |
| ProductionRGA and Peritext | Inherited Join under the embedded condition | Inherited generation policy | Reuses the underlying sequence certificate. |

Proof engineering is a separate classification.

| RDT | Proof route |
|---|---|
| Grow-only stores, flat counters, OR-Set, bounded counter, TreeMove, AegisSheet, RGA | Universal equations adapted to `CanonicalJoinLaws`. |
| Multi-value register | Native `CanonicalJoinLaws`. |
| Queue, EmbedRGA, SidedEmbedRGA, FugueMax | Direct datatype-specific `JoinAt` theorems under explicit replay-context predicates. |
| SidedPeritext | Componentwise composition with `joinAt_prod`. |
| ProductionRGA and Peritext | Inherited contextual merge certificate. |

Section 7 defines how certified execution establishes the predicate of a restricted Join theorem.

### 6.3 Ordinary adequacy

The initial configuration is canonical. Fork copies a canonical version; apply extends one with a fresh maximal event; query stutters; and merge is the Join implication after the GCA theorem identifies the base set with $E_1 \cap E_2$. Write $\mathsf{Reachable}_{\mathsf{Step}}(C_0, C)$ for the reflexive-transitive closure of the raw transition relation. Hence

$$\mathsf{Join}(D) \to \mathsf{Reachable}_{\mathsf{Step}}(\mathsf{initConfig}(D), C) \to \mathsf{HasReplayWitness}(C).$$

The proof establishes `CanonicalConfig C` first and projects replay only at the end.

*Lean:* `replayWitness_of_join`.

## 7 Issuance and client-facing correctness

### 7.1 Issuance-certified execution

An issuance policy contains

$$\mathsf{CanIssue} : \mathsf{Event}(O) \to \Sigma \to \mathsf{Prop}.$$

It constrains creation at the origin state and does not alter raw execution.

*Lean:* `Issuance`.

For a generation guard $G : \mathsf{Event}(O) \to \Sigma \to \mathsf{Prop}$, the provenance predicate is

$$\begin{aligned}\mathsf{MintHonest}(D, G, C) \iff \forall e \in C.\mathsf{events}.\ \exists \pi.\ & \mathsf{listPermOf}(\pi, \{e' \in C.\mathsf{events} \mid e'\ \mathsf{vis}\ e\}) \\ & \wedge \mathsf{respects}(\pi, \mathsf{vis}) \\ & \wedge G(e, \mathsf{applySeq}(D, \sigma_0, \pi)).\end{aligned}$$

*Lean:* `MintHonest`.

`IssuedStep D I` wraps a raw step. A non-apply label needs only its raw derivation. An apply $\mathsf{apply}(t, r, o)$ additionally requires

$$H(r) = v \wedge S(v) = (s, E) \wedge I.\mathsf{CanIssue}((t, r, o), s).$$

`MintCertifiedReach D I C` is the inductive closure from the initial configuration in which every step is issued and mint honesty is retained before and after it.

*Lean:* `IssuedStep, MintCertifiedReach`.

Certified reachability forgets to raw reachability:

$$\mathsf{MintCertifiedReach}(D, I, C) \Longrightarrow \mathsf{Reachable}_{\mathsf{Step}}(\mathsf{initConfig}(D), C).$$

Thus an all-context `Join` proof is independent of issuance: erase the issuance evidence and apply the ordinary adequacy theorem.

*Lean:* `MintCertifiedReach.toReachable`.

The second connection is explicit. For $G : \mathsf{ReplayContext}(D) \to \mathsf{Prop}$, define

$$\mathsf{IssuanceEstablishes}(D, I, G) \iff \forall C.\ \mathsf{MintHonest}(D, I.\mathsf{CanIssue}, C) \to G(R_C).$$

This predicate says only that issuance consistency establishes the configuration condition used by a restricted merge proof.

*Lean:* `IssuanceEstablishes`.

The two axes compose as follows:

$$\begin{aligned}&\mathsf{JoinOn}(D, G) \wedge \mathsf{IssuanceEstablishes}(D, I, G) \\ &\quad \to \mathsf{MintCertifiedReach}(D, I, C) \to \mathsf{HasReplayWitness}(C).\end{aligned}$$

The merge theorem supplies $G(R_C) \to \mathsf{JoinAt}(D, R_C)$; mint honesty supplies $G(R_C)$. Neither definition contains the other.

*Lean:* `replayWitness_of_mintCertified`.

### 7.2 Public interaction order

An interaction verdict is independent or conflicting. A concurrent conflict may require first-then-second, second-then-first, or no fixed order. Formally,

$$\begin{aligned}\mathsf{ConcurrentOrder} &= \{\mathsf{FstThenSnd}, \mathsf{SndThenFst}, \mathsf{Unconstrained}\}, \\ \mathsf{Interaction} &= \mathsf{Independent} \mid \mathsf{Conflict}(\mathsf{ConcurrentOrder}).\end{aligned}$$

The predicates used below are

$$\begin{aligned}i.\mathsf{Conflicts} &\iff \exists o.\ i = \mathsf{Conflict}(o), \\ i.\mathsf{FstBeforeSnd} &\iff i = \mathsf{Conflict}(\mathsf{FstThenSnd}).\end{aligned}$$

`InteractionSpec D` supplies

$$\mathsf{interaction} : \mathsf{Event}(O) \to \mathsf{Event}(O) \to \mathsf{Interaction}$$

and proves that reversing arguments flips the verdict.

*Lean:* `ConcurrentOrder, Interaction, InteractionSpec`.

Its set-relative public order is

$$\begin{aligned} e_1\ \mathsf{interactionLoOn}_{A,C,E}\ e_2 \iff\ & (e_1\ \mathsf{vis}\ e_2 \wedge A(e_1,e_2).\mathsf{Conflicts}) \\ & \vee (\neg(e_1\ \mathsf{vis}\ e_2) \wedge \neg(e_2\ \mathsf{vis}\ e_1) \wedge A(e_1,e_2).\mathsf{FstBeforeSnd} \\ & \quad \wedge \neg\exists e_3 \in E.\ e_2\ \mathsf{vis}\ e_3 \wedge A(e_2,e_3).\mathsf{Conflicts}). \end{aligned}$$

It is structurally similar to loOn but independent of the proof-local policy and concrete universal commutation.

*Lean:* `interactionLoOn`.

### 7.3 Sequential specification and public theorem

A sequential specification contains

$$\begin{aligned} \Sigma_S &: \mathsf{Type}, \\ \sigma_0^S &: \Sigma_S, \\ \mathsf{step}_S &: \Sigma_S \to \mathsf{Event}(O) \to \Sigma_S, \\ \mathsf{Legal}_S &: \mathsf{List}(\mathsf{Event}(O)) \to \mathsf{Prop}, \\ \mathsf{query}_S &: \Sigma_S \to Q \to R. \end{aligned}$$

Its run is the left fold of $\mathsf{step}_S$ from $\sigma_0^S$. Legality speaks only about the abstract history; it cannot inspect implementation state.

*Lean:* `SequentialMachine, SequentialSpec, SequentialSpec.run`.

**Definition 7.1** (Client-facing linearizability)**.** For $\mathsf{Rel} : \Sigma \to \Sigma_S \to \mathsf{Prop}$, `IsSpecLinearizable D A Spec Rel C` is

$$\begin{aligned} \forall v, s, E.\ S(v) = (s,E) \to \exists \pi.\ & \mathsf{listPermOf}(\pi, E) \\ & \wedge \mathsf{respects}(\pi, \mathsf{interactionLoOn}_{A,R_C,E}) \\ & \wedge \mathit{Spec}.\mathsf{Legal}(\pi) \\ & \wedge \mathsf{Rel}(s, \mathit{Spec}.\mathsf{run}(\pi)) \\ & \wedge \forall q.\ D.\mathsf{query}(s,q) = \mathit{Spec}.\mathsf{query}(\mathit{Spec}.\mathsf{run}(\pi), q). \end{aligned}$$

*Lean:* `IsSpecLinearizable`.

This requires a spec-legal exact history, relates implementation and abstract states, and equates every query. It is strictly the client theorem, whereas `HasReplayWitness` is an internal bridge.

## 8 Canonical virtual merge bases

An execution DAG need not have a GCA for every pair. The widened semantics supplies a synthetic base state computed from the antichain of maximal common ancestors. No synthetic base version is allocated.

### 8.1 Resolver and widened merge

A resolver $V$ contains

$$V.\mathsf{state} : \mathsf{Configuration}(D) \to \mathsf{Version} \to \mathsf{Version} \to \Sigma.$$

`StepV D V` includes every ordinary step through `base` and one additional merge constructor. For head states $s_1, s_2$, its new-version equation is

$$S'(v_m) = (\mathsf{merge}(V.\mathsf{state}(C, v_1, v_2), s_1, s_2), E_1 \cup E_2),$$

with the same head, parent, freshness, and rank updates as ordinary merge. It does not require a GCA.

*Lean:* `VirtualMergeBaseResolver, StepV, StepV.mergeVirtual`.

`IssuedStepV D V I` embeds an ordinary issued step or accepts a genuinely virtual `StepV` derivation that is not an ordinary `Step`. `MintCertifiedReachV D V I C` is its inductive closure from the initial configuration, retaining mint honesty before and after every step.

*Lean:* `IssuedStepV, MintCertifiedReachV`.

### 8.2 Recursive maximal-ancestor fold

The recursive fold needs a generalization of the pairwise definition. For a finite support $X$, let

$$\begin{aligned}\mathsf{CAWithAny}_P(X, w) &= \bigcup_{u \in X} \mathsf{CommonAncestors}(P, u, w),\\ \mathsf{IsMaximalWithAny}_P(X, w, m) &\iff m \in \mathsf{CAWithAny}_P(X, w)\\ &\quad \wedge \forall x \in \mathsf{CAWithAny}_P(X, w).\ m\ \mathsf{Reaches}\ x \to x = m.\end{aligned}$$

In other words, a candidate must be a common ancestor of $w$ and at least one member of $X$; it need not be an ancestor of every member of $X$. Lean names these definitions `CommonAncestorsWithAny` and `IsMaximalCommonAncestorWithAny`. Write

$$\mathsf{BasesWithAny}(P, X, w) = \{m \mid \mathsf{IsMaximalWithAny}_P(X, w, m)\}$$

for the finite enumeration implemented by `maximalCommonAncestorsWithAny`. The singleton bridge is

$$\mathsf{IsMaximalWithAny}_P(\{a\}, b, m) \iff \mathsf{IsMaximalCommonAncestor}(P, a, b, m).$$

*Lean:* `CommonAncestorsWithAny, IsMaximalCommonAncestorWithAny, isMaximalCommonAncestorWithAny_singleton`.

*Lean:* `maximalCommonAncestorsWithAny, mem_maximalCommonAncestorsWithAny_iff`.

Sort $\mathsf{BasesWithAny}(P, X, w)$ by increasing version rank. Let $\mathsf{stateD}(v)$ return the stored state, defaulting to $\sigma_0$ only to make the function total. The scratch fold is

$$\begin{aligned}\mathsf{vfold}(A, a, []) &= a,\\ \mathsf{vfold}(A, a, m :: ms) &= \mathsf{vfold}\big(A \cup \{m\}, \mathsf{merge}(\mathsf{vbase}(A, m), a, \mathsf{stateD}(m)), ms\big).\end{aligned}$$

The nested base is itself the fold of this sorted maximal antichain:

$$\mathsf{vbase}(X, w) = \begin{cases}\sigma_0, & \mathsf{sort}(\mathsf{BasesWithAny}(P, X, w)) = [],\\ \mathsf{vfold}(\{m_1\}, \mathsf{stateD}(m_1), ms), & \mathsf{sort}(\mathsf{BasesWithAny}(P, X, w)) = m_1 :: ms.\end{cases}$$

Termination uses the cardinality of the joint ancestor support, lexicographically paired with pending-list length. The head-pair resolver is

$$\mathsf{virtualMergeBaseState}(C, v_1, v_2) = \mathsf{vbase}(\{v_1\}, v_2).$$

The resolver whose state field is this function is $\mathsf{canonicalVirtualMergeBase}(D)$.

*Lean:* `vfoldAux, virtualBaseAux, virtualMergeBaseState, canonicalVirtualMergeBase`.

### 8.3 Virtual canonicity and adequacy

For finite $X \subseteq A_C$,

$$\mathsf{unionEvents}(C, X) = \bigcup_{v \in X} \mathsf{events}_C(v).$$

Under `StoreInv`, for allocated $w$, the maximal-base event union is exactly the outer intersection:

$$\mathsf{unionEvents}(C, \mathsf{BasesWithAny}(P, X, w)) = \mathsf{unionEvents}(C, X) \cap \mathsf{events}_C(w).$$

*Lean:* `unionEvents, maximalCommonAncestorsWithAny_unionEvents`.

The fold theorem states

$$\begin{aligned}&\mathsf{StoreInv}(S, P) \wedge \mathsf{CanonicalConfig}(C) \wedge \mathsf{JoinAt}(D, R_C)\\ &\wedge S(v_1) = (s_1, E_1) \wedge S(v_2) = (s_2, E_2)\\ &\Longrightarrow \mathsf{Canonical}(R_C, E_1 \cap E_2, \mathsf{virtualMergeBaseState}(C, v_1, v_2)).\end{aligned}$$

*Lean:* `virtualMergeBaseState_canonical`.

Consequently,

$$\begin{aligned}&\mathsf{Join}(D) \to \mathsf{Reachable}_{\mathsf{StepV}}(\mathsf{initConfig}(D), C) \to \mathsf{CanonicalConfig}(C),\\ &\mathsf{Join}(D) \to \mathsf{Reachable}_{\mathsf{StepV}}(\mathsf{initConfig}(D), C) \to \mathsf{HasReplayWitness}(C).\end{aligned}$$

*Lean:* `canonicalConfig_reachableV, replayWitnessV_of_join`.

As in the ordinary semantics, certified widened reachability erases to raw widened reachability. Hence all-context Join needs no issuance premise. The restricted form is

$$\begin{aligned}&\mathsf{JoinOn}(D, G) \wedge \mathsf{IssuanceEstablishes}(D, I, G)\\ &\quad \to \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C) \to \mathsf{HasReplayWitness}(C).\end{aligned}$$

*Lean:* `MintCertifiedReachV.toReachable`.

*Lean:* `replayWitness_of_mintCertifiedV`.

## 9 Verification certificates and packaging

A replay-adequacy certificate has field

$$\mathsf{soundV}: \quad \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C) \to \mathsf{HasReplayWitness}(C).$$

Ordinary soundness follows because ordinary execution embeds in widened execution.

*Lean:* `ReplayAdequacyCertificate, ReplayAdequacyCertificate.sound`.

The artifact provides one constructor for each merge-proof route:

$$\begin{aligned}\mathsf{Join}(D) &\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D, I),\\ \mathsf{JoinOn}(D, G) \wedge \mathsf{IssuanceEstablishes}(D, I, G) &\Longrightarrow \mathsf{ReplayAdequacyCertificate}(D, I).\end{aligned}$$

The first constructor erases certified widened reachability before applying all-context Join; the second uses issuance consistency to discharge the explicit context condition.

*Lean:* `ReplayAdequacyCertificate.ofJoin`.

*Lean:* `ReplayAdequacyCertificate.ofJoinOn`.

Let $\mathsf{CertifiedExecution}(D, I, C)$ mean either an ordinary $\mathsf{MintCertifiedReach}(D, I, C)$ derivation or a widened $\mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)$ derivation. A sequential-correctness certificate has type

$$\forall C.\ \mathsf{CertifiedExecution}(D, I, C) \to \mathsf{HasReplayWitness}(C) \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C).$$

*Lean:* `CertifiedExecution, SequentialCorrectnessCertificate`.

The complete package is

$$\begin{aligned}\mathsf{VerifiedMRDT}(D) = (&I : \mathsf{Issuance}(D),\\ &A : \mathsf{InteractionSpec}(D),\\ &K : \mathsf{ReplayAdequacyCertificate}(D, I),\\ &\mathit{Spec} : \mathsf{SequentialSpec}(D),\\ &\mathsf{Rel} : \Sigma \to \Sigma_S \to \mathsf{Prop},\\ &K_S : \mathsf{SequentialCorrectnessCertificate}(D, I, A, \mathit{Spec}, \mathsf{Rel})).\end{aligned}$$

`PackagedMRDT` additionally contains a name and the signature $D$.

*Lean:* `VerifiedMRDT, PackagedMRDT`.

Its public conclusions are

$$\begin{aligned}&\mathsf{MintCertifiedReach}(D, I, C) \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C),\\ &\quad \mathsf{MintCertifiedReachV}(D, \mathsf{canonicalVirtualMergeBase}(D), I, C)\\ &\qquad \to \mathsf{IsSpecLinearizable}(D, A, \mathit{Spec}, \mathsf{Rel}, C).\end{aligned}$$

*Lean:* `VerifiedMRDT.correct, VerifiedMRDT.correctV`.

# 10 Distributed commit-history collection

Replicas fetch commits asynchronously, so each replica decides locally when history can be removed. It must retain enough authored evidence and enough graph information for future reachability and merge-base queries.

## 10.1 Local holdings and evidence

A local holding is

$$\mathsf{Local} = (\mathsf{head} : \mathsf{Version}, \mathsf{commits} : \mathcal{P}(\mathsf{Version}), \mathsf{headPresent} : \mathsf{head} \in \mathsf{commits}).$$

An envelope contains a commit set. Advertising exports the local set; receiving unions it into the destination without changing the head. A world is $\mathsf{World} = \mathsf{Replica} \to \mathsf{Local}$.

*Lean:* `GC.Local, GC.Envelope, GC.advertise, GC.receive, GC.World`.

Authorship has type $\mathsf{Author} = \mathsf{Version} \to \mathsf{Option}(\mathsf{Replica})$. Evidence for roster member $r$ is

$$\mathsf{DerivedEvidence}(P, A, L, r) = \{v \mid v \in L.\mathsf{commits} \wedge A(v) = r \\ \wedge \mathsf{Reaches}(P, v, L.\mathsf{head})\}.$$

Evidence is complete for roster $R$ at replica $r_s$ when

$$\mathsf{EvidenceComplete}(P, A, R, r_s, L) \iff \forall r \in R.\ r = r_s \vee \\ \exists v.\ v \in \mathsf{DerivedEvidence}(P, A, L, r).$$

*Lean:* `GC.Author, GC.DerivedEvidence, GC.EvidenceComplete`.

## 10.2 Compression certificate

For any reachability relation $R_V$ on versions, define

$$\mathsf{IsGCARel}(R_V, v_1, v_2, v_T) \iff R_V(v_T, v_1) \wedge R_V(v_T, v_2) \\ \wedge \forall w.\ R_V(w, v_1) \wedge R_V(w, v_2) \to R_V(w, v_T).$$

The canonical compressed edge summarizes any old path between retained endpoints:

$$\mathsf{compressedEdge}_{P,K}(a, b) \iff a \in K \wedge b \in K \wedge \mathsf{Reaches}(P, a, b).$$

Define

$$\mathsf{CompressedReaches}_{P,K} = \left(\mathsf{compressedEdge}_{P,K}\right)^*.$$

This relation preserves reachability exactly on retained nodes. Closure of $K$ under maximal common ancestors suffices to preserve GCA answers; retaining intermediate paths or version 0 is unnecessary.

*Lean:* `GC.compressedEdge, GC.CompressedReaches, GC.compressedReaches_iff`.

*Lean:* `GC.compressed_isGCA_iff_of_maximalCommonAncestorClosed, GC.root_absent_when_dropped`.

A collection certificate chooses $K \subseteq L.\mathsf{commits}$ and proves:

1. evidence is complete, the local head is retained, and suitable evidence for every remote roster member is retained;
2. for $a, b \in K$, $\mathsf{CompressedReaches}_{P,K}(a, b)$ is equivalent to $\mathsf{Reaches}(P, a, b)$; and
3. for retained $v_1, v_2, v_T$, $\mathsf{IsGCARel}(\mathsf{CompressedReaches}_{P,K}, v_1, v_2, v_T)$ is equivalent to $\mathsf{IsGCA}(P, v_1, v_2, v_T)$.

Collection replaces the local commit set by $K$ and preserves its head.

*Lean:* `GC.Certificate, GC.collect`.

Define the pairwise closure condition by

$$\mathsf{MaximalClosed}_P(K) \iff \forall a, b \in K.\ \forall m. \\ \mathsf{IsMaximalCommonAncestor}(P, a, b, m) \to m \in K.$$

The canonical constructor then has the schematic type

$$\begin{aligned}\mathsf{ofMaximalClosed} : \mathsf{EvidenceComplete}(P, A, R, r_s, L) &\to L.\mathsf{head} \in K \\ &\to (\forall r \in R.\ r = r_s \vee \exists v \in \mathsf{DerivedEvidence}(P, A, L, r).\ v \in K) \\ &\to K \subseteq L.\mathsf{commits} \to (\forall v, p.\ p \in P(v) \to p < v) \\ &\to \mathsf{MaximalClosed}_P(K) \to \mathsf{Certificate}.\end{aligned}$$

The exact Lean constructor is named in the source anchor.

*Lean:* `GC.Certificate.ofMaximalCommonAncestorClosed`.

### 10.3 Asynchronous refinement

The collecting protocol has fetch, head synchronization, and local collection. Its comparison protocol has fetch and head synchronization but no collection. Define

$$\begin{aligned}\mathsf{StoreSim}(F, C) \iff & F.\mathsf{head} = C.\mathsf{head} \\ & \wedge C.\mathsf{commits} \subseteq F.\mathsf{commits} \wedge C.\mathsf{head} \in C.\mathsf{commits}, \\ \mathsf{WorldSim}(F, C) \iff & \forall r.\ \mathsf{StoreSim}(F(r), C(r)).\end{aligned}$$

One collecting step is matched by the corresponding network step or stuttering. Write $\mathsf{Steps}_{GC}$ and NoGCSteps for the reflexive-transitive closures of the collecting and comparison step relations, respectively. Then

$$\begin{aligned}&\mathsf{WorldSim}(F_0, C_0) \wedge \mathsf{Steps}_{GC}(C_0, C_1) \\ &\quad \Longrightarrow \exists F_1.\ \mathsf{NoGCSteps}(F_0, F_1) \wedge \mathsf{WorldSim}(F_1, C_1).\end{aligned}$$

*Lean:* `GC.StoreSim, GC.WorldSim, GC.execution_refines_noGC`.

## 11 Runtime and datatype-state collection

### 11.1 Commit-store runtime refinement

A datatype-rich runtime pairs semantics with local stores:

$$\mathsf{Runtime}(D) = (\mathsf{core} : \mathsf{Configuration}(D), \mathsf{stores} : \mathsf{World}).$$

It is well formed when each retained commit is allocated in the core and each semantic replica head agrees with its store head. A visible runtime transition contains an actual `Step` derivation and matching store evolution. Fetch and collection are silent.

*Lean:* `GC.Runtime, GC.Runtime.WellFormed, GC.RuntimeStep`.

Erasing optional labels removes silent steps:

$$\mathsf{eraseLabels} = \mathsf{List.filterMap}(\mathsf{id}).$$

Let RuntimeSteps be the list-labelled reflexive-transitive closure of runtime transitions, and let CoreSteps be the corresponding closure of raw `Step` transitions. Finite runtime traces refine raw semantic traces:

$$\mathsf{RuntimeSteps}(D, A, R, S_0, ls, S_1) \to \mathsf{CoreSteps}(D, S_0.\mathsf{core}, \mathsf{eraseLabels}(ls), S_1.\mathsf{core}).$$

The widened theorem uses the virtual-merge-base step relations and parameterizes availability by the exact versions read by the resolver.

*Lean:* `GC.eraseLabels, GC.runtime_refines_core, GC.runtime_refines_coreV`.

### 11.2 Datatype-state GC

A datatype-state protocol supplies

$$\begin{aligned} \mathsf{Physical} &: \mathsf{Type}, \\ \mathsf{semantic} &: \mathsf{Physical} \to \mathsf{Configuration}(D), \\ \mathsf{Valid} &: \mathsf{Physical} \to \mathsf{Prop}, \\ \mathsf{PhysicalStep} &: \mathsf{Physical} \to \mathsf{Option}(\mathsf{Label}(D)) \to \mathsf{Physical} \to \mathsf{Prop}. \end{aligned}$$

It proves validity preservation, silent stuttering $\mathsf{semantic}(P') = \mathsf{semantic}(P)$, and visible refinement to `StepV D V`. Write $\mathsf{SemanticSteps}_V$ for the list-labelled reflexive-transitive closure of `StepV D V`. Every finite valid physical trace therefore erases to such a widened semantic trace.

*Lean:* `StateGCProtocol, StateGCProtocol.refines`.

### 11.3 Composition

The combined physical state pairs a datatype-state physical value with a commit world. A step is either a datatype-state step, with matching store evolution for a visible label, or a silent history fetch/collection step. Write Combined.Valid for componentwise validity and CombinedSteps for the list-labelled closure of these combined steps. `combinedProtocol` is itself a `StateGCProtocol`; hence

$$\begin{aligned} &\mathsf{Combined.Valid}(P) \wedge \mathsf{CombinedSteps}(P, ls, P') \\ &\quad \Longrightarrow \mathsf{SemanticSteps}_V(\mathsf{semantic}(P), \mathsf{eraseLabels}(ls), \mathsf{semantic}(P')). \end{aligned}$$

If the datatype protocol also proves that every visible physical step is raw, the ordinary refinement follows.

*Lean:* `GC.combinedProtocol, GC.CombinedSteps.refinesV, GC.CombinedSteps.refinesRaw`.

## 12 The public theorem stack

| Layer | Contract and consequence |
|---|---|
| Operational semantics | `MRDTSig`, `Configuration`, and `Step` define the raw version-DAG machine. |
| Replay model | `ReplayContext`, loOn, and `ReplayLaws` make a finite event set denote a unique canonical fold. |
| Reachability invariant | `CanonicalConfig` says every allocated version stores that fold and carries the visibility/support facts used by induction. |
| Merge obligation | `JoinAt` maps canonical intersection and branch states to the canonical union state. `Join` and `JoinOn` specify its scope; law bundles are proof routes for all-context `Join`. |
| Internal adequacy | Reachability plus Join gives `CanonicalConfig`, then `HasReplayWitness`. |
| Client discipline | `Issuance`, `InteractionSpec`, and `SequentialSpec` independently state creation, public ordering, legality, state relation, and observations. |

| Layer | Contract and consequence |
|---|---|
| Public correctness | `VerifiedMRDT.correct` and `correctV` establish `IsSpecLinearizable`, not merely replay adequacy; `PackagedMRDT` adds the datatype name and signature. |
| Virtual bases | The recursive maximal-common-ancestor fold supplies a canonical intersection state when no GCA is available. |
| Collection | Commit-history and datatype-state GC independently refine ordinary or widened uncollected semantics and compose by trace erasure. |

`CanonicalConfig` is the inductive workhorse. `HasReplayWitness` is its internal projection, not a separate public target; `IsSpecLinearizable` is the client-facing theorem.

# A Exact correspondence index

The companion Lean ledger imports these modules and checks the named declarations, so a rename or type-level disappearance fails the document gate.

| Concept | Lean declaration | Source |
|---|---|---|
| MRDT interface | `MRDTSig` | `Signature.lean` |
| Events and internal replay support | `Op`, `UpdateSig` | `UpdateSignature.lean` |
| Replay lists | `applySeq`, `listPermOf`, `respects` | `Replay.lean` |
| Operational semantics | `Configuration`, `Label`, `Step`, `StepV` | `Execution.lean` |
| Store/GCA events | `StoreInv`, `gca_events_of_storeInv` | `StoreInvariant.lean` |
| Set-relative replay | `loOn`, `IsCanonicalState` | `SetRelativeReplay.lean` |
| Replay laws | `ReplayLaws` | `ReplayLaws.lean` |
| Merge semantics | `JoinAt`, `Join`, `JoinOn` | `MergeLaws.lean` |
| Canonical-state Join laws | `CanonicalJoinLaws`, `CanonicalJoinLaws.join`; universal adapter `CanonicalJoinLaws.ofArbitrary` | `MergeLaws.lean`; `Adequacy.lean` |
| Canonical reachability | `CanonicalConfig`, `replayWitness_of_join` | `Adequacy.lean` |
| Issuance/spec interfaces | `Issuance`, `InteractionSpec`, `SequentialSpec` | `Certificates.lean` |
| Certified adequacy | `replayWitness_of_mintCertified` | `CertifiedAdequacy.lean` |
| Client theorem | `IsSpecLinearizable`, `VerifiedMRDT` | `Correctness.lean` |
| Virtual construction | `virtualMergeBaseState` | `VirtualMergeBase.lean` |
| Virtual adequacy | `virtualMergeBaseState_canonical`, `replayWitnessV_of_join` | `VirtualAdequacy.lean` |
| History GC | `GC.Certificate`, `GC.execution_refines_noGC` | `Distributed.lean` |
| Compressed DAG | `GC.CompressedReaches` | `CompressedDAG.lean` |
| Runtime refinement | `GC.runtime_refines_coreV` | `Refinement.lean` |
| Datatype-state GC | `StateGCProtocol` | `StateGC.lean` |
| GC composition | `GC.CombinedSteps.refinesV` | `StateComposition.lean` |

# B Historical binary boundary and negative evidence

The merge-free projection $D_U$ from Section 2 is implemented by `UpdateSig`. The projection `MRDTSig.toUpdateSig` forgets queries and ternary merge, retaining only the data needed for event

application, finite replay, and commutation. It is an internal algebraic view of an MRDT, not a second datatype interface.

Some historical proof modules still study a binary operation. They request it through a separate optional capability:

$$\mathsf{HistoricalBinaryMerge}(D_U) = (\mathsf{merge}_2 : \Sigma \to \Sigma \to \Sigma).$$

Lean exposes the operation as `UpdateSig.historicalMerge`, keeping it visibly distinct from the ternary `MRDTSig.merge`. For an MRDT $D$, the compatibility instance `MRDTSig.historicalBinaryMerge` supplies the initial-state slice

$$\mathsf{merge}_2(a, b) = D.\mathsf{merge}(\sigma_0, a, b).$$

Binary merge is therefore neither a field of `UpdateSig` nor part of the live execution or `VerifiedMRDT` interfaces. It appears only when a historical theorem explicitly requests it.

*Lean:* `Foundation.UpdateSig, Foundation.HistoricalBinaryMerge, Foundation.UpdateSig.historicalMerge`.

*Lean:* `MRDTSig.toUpdateSig, MRDTSig.historicalBinaryMerge`.

Historical binary VC bundles and countermodels are not premises of the public correctness theorem. Two negative results are retained as audit controls:

1. convergence over a merely backward-closed version set can fail when the absorber search ranges over a later whole configuration; and
2. associativity, commutativity, and idempotence of binary merge do not by themselves imply the corrected contextual ternary Join property.

They are machine-checked in negative-evidence modules and remain outside the production package.

*Lean:* `Foundation.convergence_over_backward_closed_subsets_false`.

*Lean:* `Foundation.binaryLaws_insufficient`.

# C Notation and elaboration notes

- The paper writes $S$, $H$, and $L$ as partial maps. Lean totalizes them with `Option`: $S(v) = (s, E)$ abbreviates `C.ver v = some (s,E)`, and $H(r) = v$ abbreviates `C.head r = some v`.
- $f(x) = \bot$ means $x \notin \mathsf{dom}(f)$; for a totalized Lean map, this is equality with `none`.
- Lean sets have type `Set alpha`. Finiteness is normally witnessed by a list satisfying `listPermOf`, rather than stored in the set type.
- $v < v'$ in allocation rules is the natural-number rank of version identifiers. Event Lamport time is the first component of an event and is constrained separately.
- The companion ledger uses the actual Lean namespaces and checks elaboration of every principal declaration cited here.